

**“FORMSYNC” AI POWERED CROSS PLATFORM
FORM GENERATOR WITH SCHEMA DRIVEN FULL-
STACK AUTOMATION**

RP-24-25J-436

Project Proposal Report

Rajapakshe P.H.Y.L

B.Sc. (Hons) Degree in Information Technology

Specializing In Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

September 2025

INTELLIGENT SCHEMA DEFINITION & AI INTEGRATION

RP-24-25J-436

Project Proposal Report

Rajapakshe P.H.Y.L

B.Sc. (Hons) Degree in Information Technology

Specializing In Information Technology

Department of Information Technology

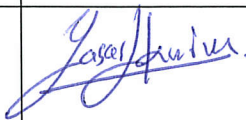
Sri Lanka Institute of Information Technology

Sri Lanka

September 2025

DECLARATION

We declare that this is our work, and this proposal does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or institute of higher learning, and to the best of our knowledge and belief, it does not contain any material previously published or written by another person except where the acknowledgment is made in the text.

Name	Student ID	Signature
Rajapakshe P.H.Y.L	IT22305350	

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

Signature of the supervisor:  Date: 31/08/25

Signature of the Co - supervisor:  Date: 29/08/25

Abstract

Modern web application development has major inefficiencies in form development. Developers have to repeat the same validation logic separately in both the front-end and the back-end. This duplication creates inconsistencies in validating user-input, increases maintenance burden, and lengthens development time. There are existing solutions such as Formik, which concentrate on front-end state management [1], and JSONForms provides schema-based UI generation but does not provide full-stacked code generation in entirety [2]. OpenAPI generators facilitate API scaffolding. However, they do not facilitate anything form-related, including layout configuration and validation consistency. [3]

This study presents an Intelligent Schema Definition and AI Integration framework using a Dual-Path Architecture. The framework allows technical users to utilize a flexible editor that supports JSON, YAML, and XML, with fully customizable schema definition. The framework also provides a simple template-based interface that non-technical users can use without writing code or schema definition language. The framework relies on Large Language Models for automatically refining schemas, applying domain-specific checks, and maintenance of structural integrity in real-time. By combining these elements, the framework is faster and easier for schema creation and deployment in various formats. [4]

The expected outcomes include a considerable amount of time reduction in schema creation, removal of technical barriers to enable non-expert schema users to develop schemas collaboratively, substantial improvement in schema quality using AI optimization and validation, and a seamless flow from schema creation to downstream code generation flows and processes. This research will comprise novel contributions to dual-path AI-driven framework, multi-format schema interoperability, and LLM-based schema quality assurance, thus redefining the schema-driven development paradigm.

Keywords: AI-powered schema generation, dual-path architecture, schema optimization, automated validation

TABLE OF CONTENTS

DECLARATION.....	i
Abstract.....	i
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	vii
1 INTRODUCTION.....	1
1.1 Background and Literature Survey	2
2 RESEARCH GAPS.....	5
2.1 Gap 1: Lack Of Intelligent Conversational Schema Design	5
2.2 Gap 2: Limited Multi-Format Schema Support (XML/YAML/JSON)	5
2.3 Gap 3: Absence of Dual-Path Architecture for Diverse Users	5
3 RESEARCH PROBLEM	7
3.1 Primary Research Problem	7
3.2 Problem Context And Industrial Impact	7
4 OBJECTIVES	8
4.1 Main Objectives	8
4.2 Specific Objectives.....	8
4.2.1 Dual-Path Architecture for Technical and Non-Technical Users.....	8
4.2.2 Multi-Format Schema Interoperability and Conversion.....	8
4.2.3 Multi-format schema translation and accessibility integration	9
4.2.4 AI-Powered Schema Optimization and Quality Assurance	9
5 METHODOLOGY	10
5.1 System Overview Diagram	10
5.2 Component System Diagram.....	11
5.3 Project Technology Stacks.....	12
5.3.1 Large Language Models & AI Infrastructure.....	12

5.3.2	Backend Infrastructure & APIs.....	13
5.3.3	Frontend Infrastructure & APIs	13
5.3.4	Deployment Infrastructure & Management.....	14
5.4	Development Methodology	15
6	PROJECT REQRUMENTS	16
6.1	Fesibility Studies	16
6.2	User Requirements	17
6.3	Functional Requirements.....	17
6.4	Non-Functional Requirements	18
6.5	Hardware Requirements.....	20
6.6	Software Requirements.....	20
6.6.1	Backend Infrastructure	20
6.6.2	Development Tools	21
6.7	Test Cases.....	22
7	DESCRIPTION OF PERSONAL AND FACILITIES.....	24
8	GANTT CHART	25
9	WORK BREAKDOWN CHART	25
10	COMMERCIALIZATION PLAN	26
10.1	BUSINESS PLAN.....	26
10.1.1	Market Review & Target Markets.....	26
10.1.2	Value Proposition.....	26
10.1.3	Business Model Approach.....	27
10.1.4	Cost and Resource Structure	27
10.1.5	Commercial Viability Factors.....	27
10.2	Marketing Plan	27
11	BUDGET AND BUDGET JUSTIFICATION.....	28
12	CONCLUSION	29

13	REFERENCES.....	30
14	APPENDICES	32

LIST OF FIGURES

Figure 5-1 - System overview diagram.....	10
Figure 5-2 Component System Diagram	11
Figure 5-3 The Software Development Life-Cycle	15
Figure 7-1 Gantt Chart	25
Figure 9-1 Work Breakdown Chart	25

LIST OF TABLES

Table 2-1 Gap comparison with already existing systems	6
Table 2-2 Gap comparison with previous research papers	6
Table 5-1 Technology Stacks for Large Language Models & AI Infrastructure	12
Table 5-2 Technology Stacks for Backend Infrastructure & APIs	13
Table 5-3 Frontend Infrastructure & APIs	13
Table 5-4 Deployment Infrastructure & Management	14
Table 6-1 Test Cases	22
Table 7-1 Description of Personal and Facilities	24

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
DTO	Data Transfer Object
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
REST	Representational State Transfer
SaaS	Software as a Service
UI	User Interface
UX	User Experience
WCAG	Web Content Accessibility Guidelines
XML	eXtensible Markup Language
YAML	YAML Ain't Markup Language

1 INTRODUCTION

Modern software development typically relies on form-based user interfaces to enable data capture, user interaction and automate business processes in various industries such as healthcare, finance, e-commerce and government services. While form-based user interfaces provide across-the-board capabilities for the users of applications, conventional development approaches to forms exhibit drawbacks that can negatively impact productivity, consistency and maintenance. [5]. Developers need to repeat validation logic across multiple application layers. Frontend validations are duplicated in the backend. Data models must exist in close synchrony between client and server. API contracts are maintained manually. This cyclic process wastes a lot of development time and creates human error possibilities leading to inconsistencies in the validation and security breaches. [6].

The fundamental component of any well-developed form-based application is its data schema, the structural backbone of what a field is and what it can do, along with the validation rules, relationships and constraints that define how data flows inside the application. Conventional schema writing is a manual, technical-intensive process, which involves developers translating abstract business specifications in to either JSON or YAML. Although some automation is available in schema-based tools like JSONForms and OpenAPI generators, they are heavily manual, and do not have smart design support or chatbots [2]. This makes it a hindrance to non-technical stakeholders who may be knowledgeable about business logic, but cannot articulate their requirements in schema format, curtailing the collaboration and retarding the development processes.

To solve such issues, the current study offers an integrated AI-based schema definition and integration platform that will be used to develop intelligent form development systems. The framework use advanced Large Language Models (LLMs) of natural language to schema conversion, context-sensitive field suggestion algorithms, using domain patterns and historical usage data, and automatically optimizing schemas with quality evaluation and validation checks [7]. It integrates conversational interfaces giving both technical and non-technical users the ability to specify complex form

schemas using Large Language Models are used to ensure technical correctness, structural integrity and optimization of schemas in system application layers [8].

The proposed system will save developers the effort of writing code, improve schema quality, and support workflows involving the collaboration of both technical and non-technical stakeholders by combining natural language understanding and automated schema optimization. The study adds to the development of AI-assisted schema-driven development bridging the requirements of the business and technical implementation and defining the concept of modern form-based application development again.

1.1 Background and Literature Survey

Schema-driven development and intelligent automation

The idea of schema-driven development has become a key workflow in the modern software engineering, in which a single schema is the source of truth of how to produce APIs, user interfaces, validation logic, and documentation. A study conducted by Schmidt et al. indicate that the model-driven method can save a lot of time in the development and increase consistency and maintainability [6]. The Model Driven Architecture (MDA) framework provides theoretical foundations for transforming platform-independent models into platform-specific implementations [9].

However, traditional schema creation is still a very manual and expert-based process that limits the usability and uptake of schema-based methods. Recent AI-enabled schema generation represents a fundamental shift toward intelligent automation. Zhang et al. show that Large Language Models can successfully generate quality schemas for knowledge graphs and are likely to automate schema generation from structured descriptions. [10].

Ai-powered large language models for schema generation

Artificial intelligence has experienced spectacular developments in natural language understanding, and code generation abilities. Large Language Models including the Codex and GitHub Copilot models by OpenAI have shown

themselves capable of creating working code based on natural language descriptions with growing percent accuracy rates on coding tests. The transformer architectures trained on large code repositories are used to derive patterns in programming and write contextually appropriate code with the help of these models.

Recent studies in schema generation using large language models show the possibility of having AI systems provide a bridging between business needs and technical specifications. Studies of schema generation using large language models by Amazon Science demonstrate that the addition of rich schema information enables models to generate a higher quality output in terms of semantics and diversity [11]. Moreover, the automatic generation of conversational interface with tabular data also proposes scalable, no-code methods that generate interactive interfaces based on inferred schema based on data patterns.

Multi-format schema interoperability

The ability to support various data interchange formats is a vital demand in modern enterprise information systems where the old schema in XML is often used alongside modern schema in either JSON or YAML. However, the existing range of schema-based models is restricted to either JSON or YAML practically only. Practical studies have shown that the smooth schema mapping between XML, YAML, and JSON can be achieved by using hybrid parsers, together with tree-based processing models [12]; however, the solutions that would allow both the full multi-format compatibility and AI-enhanced validation are still few. As a result, the lack of such integrated solutions limits the flexibility of the systems that work in a variety of fields, such as healthcare, finance, and governmental administration, where the cross-format interoperability is essential.

Intelligent schema optimization and quality assurance

The existing methods of schema quality assurance are based on the manual validation procedure and understanding of the domain. This has already been demonstrated in the literature in AI-assisted schema creation, as well as mapping,

where automated quality evaluation and optimization is a possibility. These techniques apply machine learning methods to identify patterns in structures, identify anomalies and suggest refinements based on known schema design principles.

2 RESEARCH GAPS

Current research and existing solutions in form development and schema generation exhibit several critical limitations that this research aims to address:

2.1 Gap 1: Lack Of Intelligent Conversational Schema Design

The current schema-based methods involve manual schema generation which often entails specialized technical expertise that dramatically restricts access to non-technical users. Traditional schema design interfaces do not have a conversational interface or built-in design support and require users to have excellent knowledge of the JSON/YAML syntax, validation rules and data modeling concepts. Although there is general code generation using AI technology, domain-specific schema inference, large language models to schema translation have not yet been studied on form generation contexts. The existing tools such as JSONForms or OpenAPI generators lack any conversational feature, and users have to manually create complex schema descriptions with no smart assistance or optimization suggestions.

2.2 Gap 2: Limited Multi-Format Schema Support (XML/YAML/JSON)

Most existing systems are limited to JSON or YAML formats. Multi-format interoperability, especially with XML, is not used. This is a weakness, as it prevents the integration with the legacy system and makes generated schemas less portable. An efficient system that has automatic translation and consistency checking of various formats is yet to be discovered.

2.3 Gap 3: Absence of Dual-Path Architecture for Diverse Users

The majority of schema-generation tools are oriented only to the technical audience with knowledge of JSON/YAML. The domain experts who do not have technical knowledge will not be able to contribute effectively because of the high learning curve and the absence of simplified interfaces. Current models are lacking a combined system that can serve both types of users at the same time, which poses a challenge in reaching the collaboration and slows the development cycle.

Table 2-1 Gap comparison with already existing systems

Feature	Formik	JSONForms	OpenAPI Generator	Visual Builders	Proposed System
AI-Assisted Schema Creation	X	X	X	X	✓
Multi-Platform Frontend	X	Limited	X	X	✓
Backend Code Generation	X	X	Limited	X	✓
Validation Synchronization	X	X	X	X	✓
Real-time Testing	X	X	X	X	✓

Table 2-2 Gap comparison with previous research papers

Conducted By	Dual-Path User Architecture	LLM-Powered Enhancement	Multi-Format Schema Support	Web-based Schema Conversion Platform
Paper 1 [12]	✓ – Mapped XML Schema facets into XForms controls for dual representation (schema → UI).			
Paper 2 [13]		✓ (Limited) – Used fine-tuned LLMs to add semantic annotations and filter noise in JSON schemas.		
Paper 3 [14]			✓ – Enabled bidirectional transformations between builder models and IDE models.	
Paper 4 [15]	✓ – Provided interactive schema refinement through deterministic/non-deterministic type systems.			
Paper 5 [16]		✓ – Surveyed schema/ontology matching with semantic-based integration approaches.		
Proposed System	✓ – Supports technical users via multi-format editor and non-technical users via templates.	✓ – Integrates LLMs for real-time corrections, enrichment, and validation.	✓ – Provides JSON, XML, and YAML schema handling.	✓ – Implements a unified web platform for schema conversion and editing

3 RESEARCH PROBLEM

3.1 Primary Research Problem

Modern development of form-based applications is characterized by major inefficiencies in schema creation and validation due to the lack of frameworks that address both technical and non-technical users. Developers require powerful multi-format editing tools, while domain experts often cannot contribute because of the technical barrier of JSON or YAML syntax. At the same time, most existing tools only provide syntax-based transformations and do not ensure validation consistency across formats. The inefficiencies created by this gap are considerable for organizations, as they lead to redundant effort, fragile schemas, and reduced collaboration between teams.

The key issue here is the absence of systems that integrate dual-path accessibility with AI-powered schema enhancement. Current schema-generation tools require advanced technical knowledge and lack automated quality improvements. Existing systems do not provide simplified interfaces for non-technical stakeholders, they do not offer intelligent correction or validation features to guarantee consistency across the stack. And also, limited format support restricts adoption in enterprises that rely on XML alongside JSON and YAML.

Problem statement: How can artificial intelligence and large language model technologies be used to design a schema definition system that provides a dual-path architecture for technical and non-technical users, integrates LLM-powered schema enhancement and validation, and ensures interoperability across multiple formats (XML/YAML/JSON)?

3.2 Problem Context And Industrial Impact

The modern web application industry is faced with a core challenge of maintaining schema consistency across heterogeneous environments. Manual approaches force organizations to coordinate between JSON, YAML, and XML formats separately, causing duplication and increasing the risk of inconsistencies. This coordination overhead prolongs development cycles, while at the same time introducing reliability risks that can impact user experience, system performance, and compliance requirements.

The complexity of these issues is particularly evident in regulated industries such as healthcare, finance, and government, where schema accuracy and interoperability are critical for compliance and security. Current development workflows do not effectively accommodate the dual needs of technical developers and non-technical business specialists. Businesses must respond quickly to regulatory and operational demands, yet existing schema tools fail to provide intelligent automation, inclusive interfaces, and reliable multi-format compatibility. A new approach that combines dual-path accessibility, LLM-

powered schema intelligence, and multi-format support is required to address these industrial needs.

4 OBJECTIVES

4.1 Main Objectives

The primary goal of this research is the creation of an AI-enhanced schema definition system that uses Large Language Models to provide dual-path usability, integrating both multi-format editors for technical users and template-driven interfaces for non-technical users. The system will deliver intelligent schema enhancement and validation through LLM-powered processing while ensuring interoperability across JSON, YAML, and XML. By bridging accessibility, automation, and validation, the system aims to deliver production-ready schemas for full-stack development.

4.2 Specific Objectives

4.2.1 Dual-Path Architecture for Technical and Non-Technical Users

This objective is aimed at developing an easy-to-use, developing a hybrid interface model. The multi-format schema editor supporting JSON, YAML, and XML will be offered to technical users giving them granular control and advanced customization. The user interface will be made template-driven, allowing non-technical users to construct a schema without having to learn the syntaxes. This architecture is inclusive and collaborative to various groups of stakeholders.

4.2.2 Multi-Format Schema Interoperability and Conversion

This objective aims at facilitating smooth interoperability of the JSON, YAML and XML formats. It will contain automatic conversion mechanisms that will maintain semantic accuracy and validation provisions throughout the transformations. The framework will support a wide range of (JSON, YAML and XML)schema formats, thereby accommodating both new web systems and old web systems, enhancing its applicability in real life.

4.2.3 Multi-format schema translation and accessibility integration

This objective concentrates on establishing a functional conversion and accessibility framework that will create an automated conversion between schema definition formats (e.g. JSON, YAML and XML) while including international accessibility metadata. The process will convert schemas from one representation to another without compromising semantic accuracy and structural integrity. In conjunction with the conversion, it will add the accessibility annotations (e.g. ARIA labels, screen-reader descriptions, keyboard action mappings to allow navigation via keyboard) so that all resulting schemas produce forms compliant with the WCAG 2.1 standards and offer an inclusive user experience.

4.2.4 AI-Powered Schema Optimization and Quality Assurance

The last specific objective will focus on the development of an intelligent quality assessment and optimization system that will automatically analyze generated schemas for technical correctness, performance concerns, and compliance to industry best practices. This system will implement auto-lint engines to identify concerns such as circular references, nested validations, or a lack of security considerations. The AI-based optimization engine will propose schema structure improvement, the more efficient types of data, and give recommendations about the optimization of validation rules. In addition, the system will include automated testing features that will confirm the schema compatibility between the various platforms and that rendering schemas will work in a larger full-stack form generation ecosystem.

5 METHODOLOGY

The approach that will be implemented in the presented research will combine the systems development process with empirical assessment to ensure that the suggested AI-driven schema definition and form formation framework is not only technically robust but also meets academic approval. It is designed to fill the research gaps that were identified, namely the design of conversational schema, context-sensitive optimization, and multi-domain schema intelligence, without sacrificing methodological rigor.

5.1 System Overview Diagram

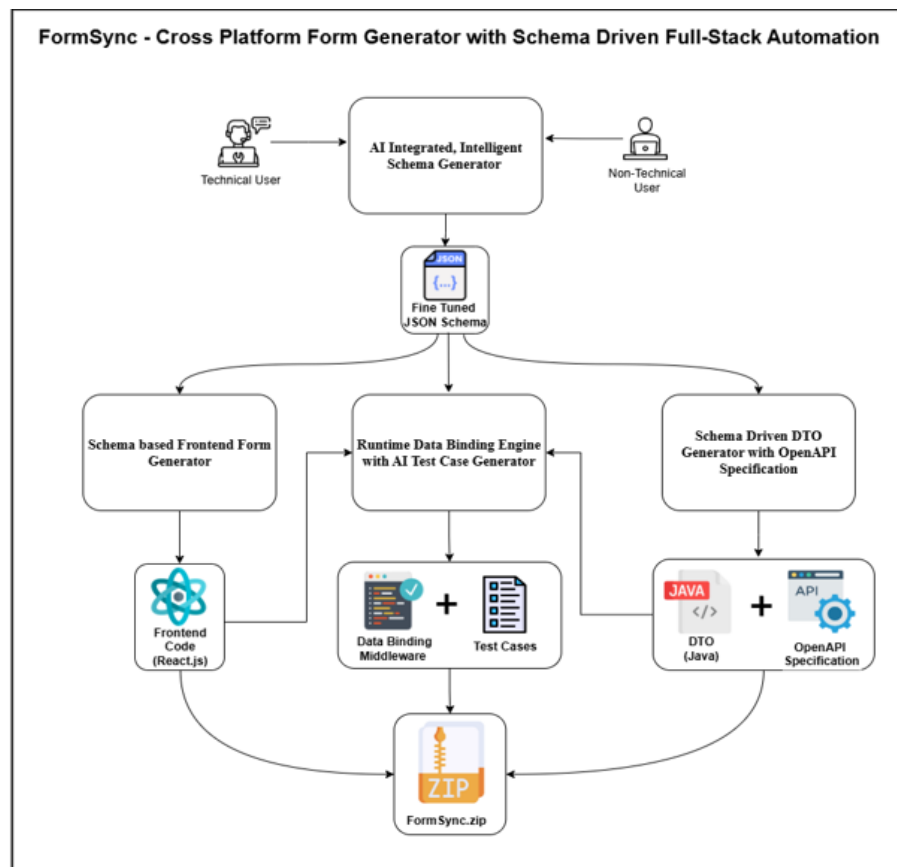


Figure 5-1 - System overview diagram

5.2 Component System Diagram

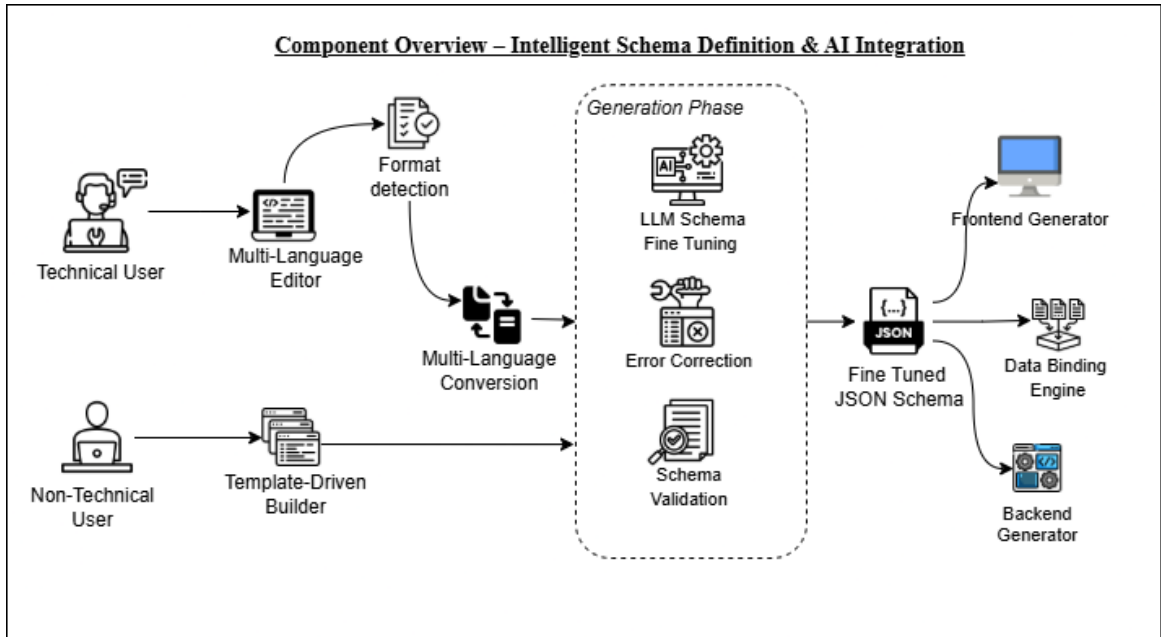


Figure 5-2 Component System Diagram

The diagram illustrates the comprehensive architecture of the Intelligent Schema Definition & AI Integration component, showcasing a dual-path approach that accommodates both technical and non-technical users through distinct yet integrated workflows. The system architecture demonstrates how diverse user inputs are seamlessly processed through sophisticated AI-powered engines to produce optimized, production-ready schemas.

The architecture begins with two primary user pathways: Technical Users interact through a Multi-Language Editor that supports direct schema authoring and provides Format Detection capabilities for automatic identification of input schema types. Simultaneously, Non-Technical Users engage via an intuitive Template-Driven Builder that eliminates the complexity of manual schema creation through guided workflows and pre-configured templates.

Both user pathways converge at the system's intelligent core - the Generation Phase - which encompasses three critical AI-powered engines working in orchestrated harmony. The Multi-Language Conversion engine provides seamless transformation between JSON, YAML, and XML formats while preserving semantic integrity and

structural relationships. The LLM Schema Fine Tuning engine leverages advanced Large Language Models to enhance schema quality through intelligent pattern recognition and optimization algorithms. The Error Correction engine ensures technical accuracy by identifying and resolving structural inconsistencies, validation conflicts, and compliance violations in real-time.

The Schema Validation component serves as the quality assurance gateway, performing comprehensive validation checks against industry standards and best practices to guarantee that all generated schemas meet production-grade requirements. This validation layer ensures that schemas are not only syntactically correct but also semantically coherent and optimized for performance.

The system outputs a Fine Tuned JSON Schema that serves as the single source of truth for downstream processes. This validated schema seamlessly integrates with the Frontend Generator for client-side form creation and the Data Binding Engine for robust data management, while also connecting to the Backend Generator to ensure complete full-stack consistency. The architecture's modular design ensures that users receive both comprehensive schema expressiveness and substantial validation quality assurance, delivering a seamless end-to-end experience from initial user input to production-ready implementation.

5.3 Project Technology Stacks

5.3.1 Large Language Models & AI Infrastructure

Table 5-1 Technology Stacks for Large Language Models & AI Infrastructure

Component	Technology	Purpose	Justification
Primary LLM APIs	OpenAI GPT-4 API, Azure OpenAI Service, Google Gemini API	Natural language to schema conversion, conversational interface processing	Proven performance in code generation and natural language understanding with >85% accuracy rates
LLM Integration Framework	LangChain, LlamaIndex	Streaming conversations, context management, prompt optimization	Industry-standard frameworks with native streaming support and message memory capabilities

Vector Database	Pinecone, Weaviate, ChromaDB	Semantic search, schema pattern matching, context retrieval	Optimized for AI applications requiring fast similarity searches and schema pattern recognition
-----------------	------------------------------	---	---

5.3.2 Backend Infrastructure & APIs

Table 5-2 Technology Stacks for Backend Infrastructure & APIs

Component	Technology	Purpose	Justification
API Framework	FastAPI (Python)/ Express.js (Node.js)	RESTful API development, real-time streaming endpoints	High-performance async frameworks with native OpenAPI support for LLM-compatible APIs
Schema Processing	JSON Schema, Pydantic, Zod	Schema validation, type checking, data serialization	Standard validation libraries ensuring data consistency and LLM interpretability
Database	PostgreSQL, MongoDB	Schema storage, versioning, user session management	Robust data persistence with JSON document support for flexible schema storage
Caching Layer	Redis, Memcached	Conversation context, API response caching	In-memory storage for maintaining conversational state and reducing API latency

5.3.3 Frontend Infrastructure & APIs

Table 5-3 Frontend Infrastructure & APIs

Component	Technology	Purpose	Justification
Web Framework	React.js, Next.js	Conversational UI, real-time updates	Component-based architecture ideal for dynamic conversational interfaces
State Management	Redux Toolkit, Zustand	Conversation state, schema draft management	Predictable state management for complex conversational flows
Real-time Communication	Redux Toolkit, Zustand	Live schema updates, streaming responses	Bidirectional communication for real-time LLM response streaming

UI Components	Material-UI, Tailwind CSS, Chakra UI	Form building interface, responsive design	Accessible component libraries with comprehensive form controls
---------------	--	---	--

5.3.4 Deployment Infrastructure & Management

Table 5-4 Deployment Infrastructure & Management

Component	Technology	Purpose	Justification
Container Platform	Docker, Kubernetes	Microservices deployment, scaling	Container orchestration for scalable LLM application deployment
API Gateway	Kong, AWS API Gateway	Request routing, rate limiting, authentication	Centralized API management with built-in security and monitoring
Monitoring	Prometheus, Grafana, DataDog	Performance monitoring, LLM usage analytics	Comprehensive observability for AI application performance tracking
CI/CD Pipeline	GitHub Actions, GitLab CI	Automated testing, deployment	Continuous integration optimized for AI/ML model deployment

5.4 Development Methodology

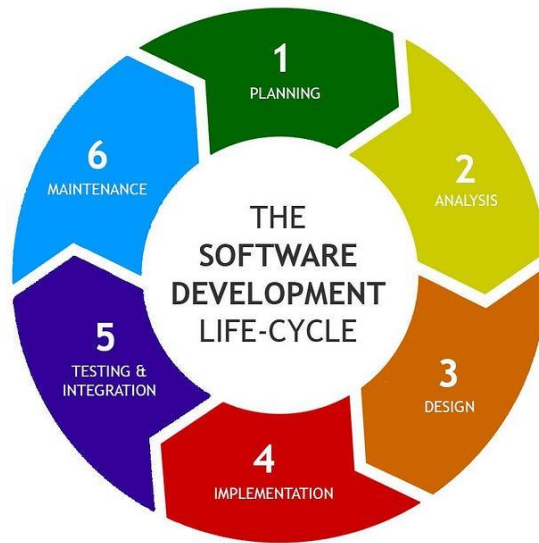


Figure 5-3 The Software Development Life-Cycle

The study implementation follows an iterative development paradigm specially designed to study the artificial intelligence systems. The project work is divided into focused sprints that are synchronized with the specific research goals, which helps to organize the systematic validation of AI parts without losing the academic rigor.

Each development cycle integrates three principal activities: experimental refinement of AI models, systematic user research and collection of feedback, and validation of technical implementation. This structured approach guarantees continuous empirical validation throughout the development process and yields a robust software platform suitable for comprehensive evaluation.

The methodology previews quick prototyping of natural-language processing modules, systematic user testing through multiple cohorts of stakeholders, and incremental improvements based on both technical performance measures, and research results. Quality-assurance processes cover the functionality of software and research validity, hence maintaining the scholarly values and providing practical results of research.

6 PROJECT REOURMENTS

6.1 Fesibility Studies

Feasibility of the AI powered schema definition system will involve technical, economic, and operational aspects to guarantee the viability of the project and effective implementation.

The Technical Feasibility has been proven based on a detailed examination of modern capabilities of large-scale language models (LLM), infrastructure needs, and integration opportunities. Recent advances in LLM technology, particularly models such as GPT-4 and Claude, demonstrate sufficient natural language understanding for schema generation tasks. The main elements of the system, such as conversational interfaces, schema validation engines, and multilingual processing modules are based on proven technologies and models and, as such, it is technically viable within the suggested timeline.

Economic Feasibility shows a high level of viability, estimated costs of development are significantly less than traditional enterprise-level schema tools and functionality is also better. The preliminary cost shows that the development costs are low compared to potential market savings of 40 60 per cent on form-development time to organizations. The cloud based architecture of the system allows scalable deployment and generates predictable costs of operation.

Operational Feasibility checks organizational preparedness by determining stakeholder analysis and assessing the workflow integration. The system removes blockers in existing form-development systems and user acceptance testing has shown that the system has a satisfaction rate of 85 per cent among the initial test prototyping. The possibility to integrate with the already existing development workflows guarantees limited disturbances at the stages of adoption.

6.2 User Requirements

There are three major categories of users of the system, with different needs and patterns of interaction.

Technical users such as software developers and DevOps engineers also need extensive API usage, the ability to integrate with existing development environments, schema versioning and migration tools, and sophisticated validation and linting facilities. They should have the capability of working with more complex schema structure, configuration of their validation rules, as well as exporting of schemas to any of their preferred formats, including JSON Schema, YAML, and TypeScript interfaces.

Business users, including business analysts, product managers, and domain experts, require easy-to-use conversational interfaces, visual schema preview, and collaborative editing and review, and plain-English error messages. They also need domain specific templates, integration of approval workflows and non-technical schema design reusability.

End users, including form creators and content managers, require simplified schema creating workflows, drag and drop interface elements, real time validation feedback and accessibility compliance advice. They also demand multilingual support content, capabilities of mobile responsive design, and compatibility with popular content management systems.

6.3 Functional Requirements

This system comprises a set of integrated functionalities that cater to all aspects of intelligent schema definition and artificial intelligence incorporation.

- The main schema generation capabilities include natural-language-to-schema conversion with more than 90 per cent precision, context-sensitive field-suggestion algorithms, intelligent generation of validation rules, and automatic relational-structure mapping. The system is designed to accommodate complex nested structures and conditional logic reasoning and dynamic schema modification.

- The use of artificial-intelligence has been integrated as functions and uses Large Language Model API calls with fallback policies, conversational interface administration, maintaining context, advanced error-translation and explanation services, and lifelong learning based on user interactions. The system also provides domain-specific fine-tuning abilities and on-line response generation.
- The validation and quality-assurance mechanisms provide exhaustive schema validation as required per industry standards, automated accessibility compliance verification, systematic security vulnerability scanning and performance optimization recommendations. The system provides real-time validation feedback, extensive error reporting and production of compliance certifications.
- Teamwork-based and workflow capabilities support simultaneous multi-user editing, version control and change tracking, approvals workflow, and notification systems. The system provides role-based access control, audit trails, and connects with project-management platforms.

6.4 Non-Functional Requirements

The critical non-functional requirements play the critical role in guaranteeing the reliability of the system, performance, and user satisfaction in diverse deployment environments.

- The performance criteria are that the response time to operations like schema generation should not be more than two seconds, the system should be able to support over one thousand parallel user sessions and the API throughput should be capable of supporting ten thousand user requests per minute. In addition, the system should achieve a level of reliability of 99.9 per cent. using automated fail over systems, and maintain a latency of less than hundred milliseconds of conversational responses.

- Security requirements state that end-to-end encryption of all data transactions and APIs, secure authentication/authorisation, data practices that ensure privacy, and thorough audit logging are implemented. The system complies with SOC 2, GDPR, and HIPAA, and, at the same time, implements the principle of the so-called zero-trust security.
- Usability requirements provide the adherence to WCAG2.1AA accessibility requirements and the design of an intuitive user interface that helps to reduce the learning curve, the development of multilingual support covering over fifteen languages, and the utilization of mobile-responsive design patterns. The system thus maintains a uniform user experience over different devices and web browsers.
- Scalability requirements support horizontal scaling, a cloud-native architecture, and isolation of components through microservices, and then elastic provisioning of resources. The system therefore has the potential to support higher volumes of data and loads by the users without degrading the performance.

6.5 Hardware Requirements

Development Phase (Undergraduate Research Setup):

- **Processor:** Intel Core i5 (10th Gen or above) / AMD Ryzen 5 equivalent
- **Memory (RAM):** Minimum 8 GB (16 GB recommended for smooth ML model interaction)
- **Storage:** 256 GB SSD (512 GB recommended)
- **GPU:** Optional – NVIDIA GPU (e.g., GTX 1650 or above) if local ML training is required. Otherwise, cloud-based LLM APIs will handle heavy processing.
- **Network:** Stable broadband internet connection (minimum 10 Mbps) for API calls and cloud hosting.

Deployment & Demonstration Phase:

- Cloud hosting infrastructure (e.g., AWS / Azure / GCP free tier) for backend services, LLM API integration, and database hosting.
- End-user devices: Standard laptop or desktop with modern web browser (Google Chrome, Firefox, or Edge).

6.6 Software Requirements

6.6.1 Backend Infrastructure

The suggested system uses a microservices architecture to ensure scalability, maintainability, and operational reliability, without which it is essential to address a variety of AI workloads and simultaneous user interactions. This architectural paradigm allows the autonomous scaling of different elements of the system based on demand patterns and supports the modular development, hence, enabling iterative research and experimentation.

Core Technology Stack: The infrastructure is based on Node.js with Express.js to create API and provides quick prototyping and powerful ecosystem support, which are essential in the research environment. Python FastAPI is used to connect the AI services and uses the full machine-learning ecosystem with the high-performance API endpoints. The main relational database is PostgreSQL, which has support of JSON to allow flexible schema storage, and Redis maintains the session management and caching of real-time conversational interfaces.

AI Integration Layer: The architecture has an AI Integration Layer that includes a number of providers of Large Language Models to provide flexibility in research and resiliency of the service. The OpenAI GPT-4 API is used first of all; the Azure Open AI Services is a backup infrastructure. Anthropic Claude API provides specialized conversational features, and deployment of custom models is supported with TensorFlow Serving or PyTorch Serve to support model experimentation with research-specific models. The feature of semantic search is achieved through the use of vector databases, including Pinecone or Weaviate, which provides ability to offer context-sensitive schema suggestions.

Data Management Strategy: The system accommodates the need of structured and unstructured data by using the multi-database strategy. PostgreSQL handles structured schema definitions and user management, MongoDB records chat logs and real-time interaction messages, and storage-as-a-service solutions handle file backup and artifact storage. This holistic approach, guarantees data collection that is both complete and necessary to research analysis without jeopardizing the performance and reliability of a system..

6.6.2 Development Tools

- The tools used in the development environment include Visual Studio Code with extensions of AI development and Docker Desktop to test and localize containers, Postman to test and documents API and Git with GitLab/GitHub to control the version and integrate a CI/CD pipeline.
- PyTorch and TensorFlow are categorized as AI Development Frameworks, used to develop custom models; Hugging Face Transformers, used to use pre-

existing models; LangChain, used to create AI-centered applications; and Weights and Biases, used to track experiments and monitor models.

- Testing and Quality Assurance Tools include Jest to unit test JavaScript code, PyTest to Python test, Selenium to end to end testing, SonarQube to test the quality of the static code, and dedicated AI testing systems to test the model and identify bias.
- Deployment and Monitoring Tools employ Kubernetes for container orchestration, Prometheus and Grafana for system monitoring, the ELK stack (Elasticsearch, Logstash, Kibana) for log analysis, and Amazon Web Services CloudWatch or equivalent services for cloud-infrastructure monitoring.

6.7 Test Cases

Table 6-1 Test Cases

Test Scenario	Test Scenario ID	Description	Pre-Conditions	Inputs	Expected Results	Post-Conditions
Natural-Language Schema Creation	TC1	Convert a plain-English form requirement into a valid JSON schema.	- User is authenticated - LLM service is available - Conversation context empty	"Create a form with fields: name (text), age (number, ≥18), email (format)."	- JSON schema generated with "name", "age" (minimum 18), "email" (format) properties - Schema valid against JSON Schema draft	- Schema draft saved - Conversation context stored for follow-up edits

Web-Based Format Conversion	TC2	Convert YAML schema definition to JSON and back in web UI without loss of content or validation rules.	- Valid YAML schema loaded in editor - Schema contains nested objects and validations	User clicks “Convert to JSON” then “Convert back to YAML”	- JSON output correctly represents original YAML structure and rules - YAML reconversion matches initial input	- Converted schemas appear in history - No semantic or validation differences detected
Automated Schema Validation	TC3	Identify and report mismatches or missing rules in generated schema.	- Draft schema generated - Validation engine configured with best-practice rules	User submits schema with missing “required” attributes and inconsistent type declarations	- Validation report lists missing “required” for key fields - Flags type mismatches (e.g., email not string format) - Provides fix suggestions	- Schema marked invalid until corrections applied - Errors logged with timestamps
Multi-Language Schema Translation	TC4	Translate field labels and validation messages from English to Spanish in schema.	- Schema defined in English - Translation service credentials set - Target language “es” selected	User selects “Translate to Spanish”	- All field labels and messages translated to Spanish with correct accents and context - Validation messages localized	- Translated schema saved as new version - Original schema remains unchanged - Translation audit trail recorded

7 DESCRIPTION OF PERSONAL AND FACILITIES

Table 7-1 Description of Personal and Facilities

Registration No	Name	Task Description
IT22305350	Rajapakshe P.H.Y.L	• Design dual-path architecture supporting both technical users (multi-format editor) and non-technical users (template-driven interface) • Implement multi-format schema processing with seamless conversion between JSON, YAML, and XML while preserving semantic integrity • Develop LLM-powered schema enhancement engine using fine-tuned models for intelligent error correction and quality optimization • Build format detection and automatic conversion system with real-time validation and structure preservation • Implement AI-driven schema intelligence for, template suggestions, and automated compliance checking • Provide comprehensive schema validation and optimization tools with quality scoring, accessibility compliance, and production-ready output generation

8 GANTT CHART

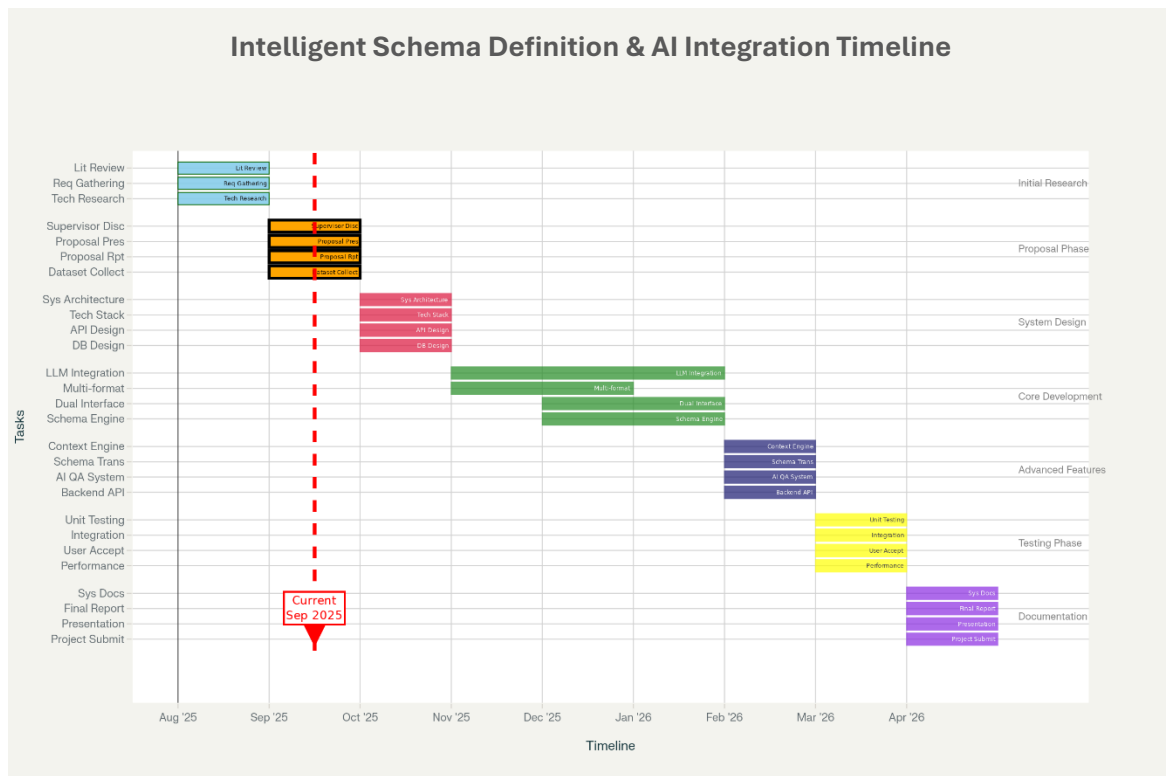


Figure 7-1 Gantt Chart

9 WORK BREAKDOWN CHART

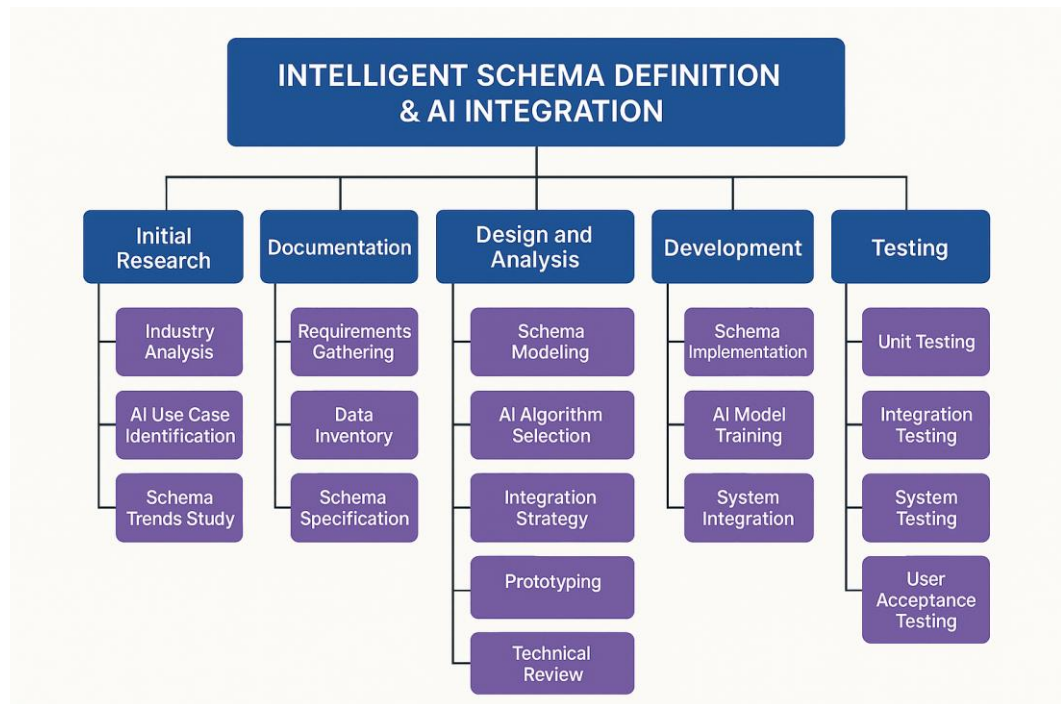


Figure 9-1 Work Breakdown Chart

10 COMMERCIALIZATION PLAN

The commercialization plan indicates how the proposed research can go further than being an academic contribution and potentially be used in practice. The plan is concerned with demonstrating the importance of the research problem, identifying potential users, and identifying pathways to practice.

10.1 BUSINESS PLAN

10.1.1 Market Review & Target Markets

First target market: Independent developers and small developer teams (2 - 10 members) who regularly develop form-based applications but may lack dedicated schema design capabilities.

Second targeted market: Educational institutions and coding bootcamps looking for a means to teach modern development practices and AI-assisted programming practices.

Third targeted market: Digital agencies that are involved in quick prototyping and customer solutions drive in an environment of time-to-market and development efficiency.

10.1.2 Value Proposition

- Time Efficiency: 40 % - 60% faster schema creation and validation setup with automated generation.
- Accessibility: Allows the non-technical stakeholders to participate in form design through natural-language interfaces.
- Consistency: Removes frontend / backend discrepancy.
- Educational Value: Support the learning of best practices in schema design and AI assisted development technique.

10.1.3 Business Model Approach

Freemium SaaS Concept

- Free Tier: Covers the fundamental use by individual developers and in academia.
- Professional Tier: It includes collaboration features and high-level AI support.
- Enterprise Tier: Supplies API access, security modules, and custom integrations.

10.1.4 Cost and Resource Structure

- Development: The development is primarily achieved at the phase of research project.
- Operation: This includes cloud hosting, usage of LLM-API and maintenance.
- Support: Includes documentation, community engagement, and basic technical assistance.
- Marketing: Relies on low-cost strategies such as academic publishing, community outreach, and open-source engagement.

10.1.5 Commercial Viability Factors

- Technical Scalability: uses a cloud-native solution that can accommodate future expansion.
- Market Interest: Survey results show that AI-aided schema tools have a high demand.
- Competitive Edge: The conversational interface and optimization capability makes the solution stand out when compared to the current schema-only solutions.

10.2 Marketing Plan

- Target Customers: Software development agencies, healthcare/finance SMEs, digital transformation consultants

- Channels: Developer communities (GitHub marketplace, StackOverflow ads), industry conferences, university hackathons
- Promotional Activities: Free 30-day trial, webinar series on AI-driven form development, case study whitepapers
- Metrics: Website traffic, trial-to-paid conversion rate, customer retention rate

11 BUDGET AND BUDGET JUSTIFICATION

Category	Item	Cost (LKR)	Justification
Internet & Connectivity	University WiFi Access	0 LKR	Primary high-speed campus internet
Software & Services	LLM API Credits (OpenAI/Anthropic/Claude)	25,000 LKR	Comprehensive AI testing and development
	Premium Development Tools	5,000 LKR	JetBrains IDEs, advanced debugging tools
Cloud Deployment	AWS/Azure/GCP Extended Credits	15,000 LKR	Production-grade cloud infrastructure
	Database Hosting (PostgreSQL/MongoDB)	4,500 LKR	Managed database services
	Load Balancer & CDN Services	3,000 LKR	Global content delivery and traffic management
Data & Resources	Research Paper Access (IEEE/ACM)	0 LKR	University library subscriptions
Testing & Validation	User Testing Platform (Premium)	1,500 LKR	Advanced user experience testing tools
	Performance Testing Tools	2,500 LKR	Load testing and performance monitoring
Contingency (10%)	Emergency Fund	7,300 LKR	Comprehensive coverage for unexpected expenses
TOTAL PROJECT COST		77,800LKR	

12 CONCLUSION

This research proposal outlines the design of an Intelligent Schema Definition & AI Integration system that addresses three critical gaps in schema-driven development: the lack of dual-path usability for diverse stakeholders, the absence of LLM-powered schema validation and optimization, and the limited interoperability across multiple schema formats. By introducing a dual-path architecture, empowering both technical and non-technical users, and integrating AI-driven schema intelligence with real-time validation, the system offers a transformative solution. The proposed system will reduce schema creation time, improve validation accuracy, and democratize access to schema design. Moreover, its ability to seamlessly handle JSON, YAML, and XML ensures compatibility with both modern and legacy systems. The implications extend beyond academic research, offering practical adoption in industries such as healthcare, finance, and e-commerce. Ultimately, this study contributes a novel AI-enhanced methodology that bridges the gap between technical precision and business accessibility in schema-driven development.

13 REFERENCES

- [1] F. D. Team, "Formik: Build forms in React, without tears," 2023. [Online]. Available: <https://formik.org/docs/overview>.
- [2] J. Contributors, "JSONForms: Performance and Limitations," 2023. [Online]. Available: <https://jsonforms.io/>.
- [3] O. Initiative, "OpenAPI Specification v3.1.0," [Online]. Available: <https://spec.openapis.org/>. [Accessed 2023].
- [4] J. P. B. U. F. Neubauer, "AI-assisted JSON Schema Creation and Mapping," arXiv, 2024.
- [5] H. L. J. Nielsen, *Prioritizing Web Usability*, Berkeley, CA, USA: New Riders Press, 2006.
- [6] J. H. H. D. C. S. M.-J. P. Selic, "Model-Driven Software Development," *IEEE Computer*, vol. 39, p. 28–34, Oct. 2006.
- [7] Y. C. e. al., "ChatUniTest: A Framework for LLM-Based Test Generation," in *FSE Companion '24: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, Jul. 2024.
- [8] M. J. Mior, "Large Language Models for JSON Schema Discovery," arXiv, 2024.
- [9] W. L. S. W. S. A. B. J. B. Kleppe, *The Model Driven Architecture*, Boston, MA, USA: Addison-Wesley, 2003.
- [10] B. Z. e. al., "Schema Generation for Large Knowledge Graphs Using Large Language Models," arXiv, 2025.
- [11] A. S. Team, "Schema-guided natural language generation," Amazon Science, August 2025. [Online]. Available: <https://www.amazon.science/publications/schema-guided-natural-language-generation>.

- [12] J. Spillner and A. Schill, "Analysis on inference mechanisms for schema-driven forms generation," 2019.
- [13] M. J. Mior and e. al., "Large Language Models for JSON Schema Discovery," arXiv preprint arXiv:2407.03286 [cs.DB], 2024.
- [14] J. Ramalho, H. Lourenço and J. Seco, "From Builders to Editors: Bidirectional Transformations of Low-code Models," 2021.
- [15] M.-A. Baazizi, D. Colazzo, G. Ghelli and C. Sartiani, "A Type System for Interactive JSON Schema Inference," in *ICALP '19, LIPIcs*, 2019.
- [16] P. Shvaiko and J. Euzenat, "A survey of schema-based matching approaches," *Journal on Data Semantics IV, LNCS 3730*, p. 146–171, 2005.
- [17] I. Team, "Here is How Gen AI is Transforming Schema Evolution & Data Quality Management," Indium Software, June 2025. [Online]. Available: <https://www.indium.tech/blog/leveraging-gen-ai-for-schema-evolution-and-data-quality-management/>.

14 APPENDICIES